

Detecting Prompt Injection Across RAG and MCP Surfaces

Phase 2 Report: Literature Review and Data Acquisition

Carter Corbin Max Edmonds Mohammed Adil Ahmed
Oluwamayowa Adewummi Adedapo Odeyemi

College of Computing and Software Engineering
Kennesaw State University, Kennesaw, GA

CS 7267 Machine Learning · October 2026

Abstract

LLM applications increasingly read text that their users never wrote: passages retrieved for retrieval-augmented generation (RAG), documents fetched by tools, and the descriptions and responses of Model Context Protocol (MCP) tools. Any of it can carry an indirect prompt injection. This report covers Phase 2 of our project, which builds a detector for such untrusted text. We review 20 papers on attacks, defenses, benchmarks and evaluation methodology, code them on a shared frame, and organize them into a taxonomy by attack surface, payload and defense class. Within the reviewed set, no defense targets MCP tool descriptions, no detector targets tool outputs, and no detector is evaluated on an attack surface it never saw in training. We then describe the corpus built to study those gaps: 11,005 labeled text segments from four public benchmarks, versioned with DVC, split by group with two sources held out entirely for leave-one-surface-out and cross-benchmark evaluation, profiled for nulls and class balance, and documented in a data card. The profile also exposes a length shortcut that every detector in Phase 3 must be measured against.

Contents

1	Introduction	3
2	Literature Review	3
2.1	Corpus and coding protocol	3
2.2	Coverage of the reviewed set	4
2.3	Taxonomy	5
2.4	State-of-the-art benchmarks	6
2.5	Gaps this project addresses	7
3	Data Acquisition	8
3.1	Sources and licensing	8
3.2	Download and versioning	8
3.3	Construction and cleaning	9
3.4	Profile	10
3.5	Licensing, privacy and IRB	11
3.6	Data card	11
4	Summary and Next Steps	12
A	Data card	12
A.1	At a glance	12
A.2	Motivation	13
A.3	Composition	14
A.4	Collection process	17
A.5	Preprocessing, cleaning and labeling	18
A.6	Uses	18
A.7	Distribution	19
A.8	Maintenance	19
	References	20

1 Introduction

An LLM application that retrieves documents or calls tools places untrusted text in the model’s context. An attacker who controls that text, for example a web page, an email, a passage in a retrieval corpus or the description of a third-party tool, can plant instructions or false content without ever interacting with the user. Greshake et al. named this threat *indirect prompt injection* and showed it against deployed systems [1]. The Model Context Protocol (MCP) widens the attack surface further: tool descriptions enter the agent’s context when a server is registered, before any tool runs [2], [3].

Research question. Can a detector that classifies untrusted text segments as adversarial or benign generalize (i) within the surfaces it is trained on, (ii) to an attack surface it has never seen, and (iii) to a benchmark it has never seen?

Phase 2 lays the groundwork: a structured review of the state of the art, and a dataset built and documented to answer the question. Table 1 maps each Phase 2 requirement to the section and repository artifact that addresses it.

Table 1: Phase 2 requirements and where this report and the repository address them.

Requirement	Where it is addressed	Repository artifact
Review 20–30 papers	§2.1, §2.2	Coding matrix (<code>literature-review/*.xlsx</code>)
Organize into a taxonomy table	§2.3, Tables 2–3	<code>literature-review/tables.md</code> (generated)
Identify SOTA benchmarks	§2.4, Table 4	<code>literature-review/tables.md</code>
Note gaps the project addresses	§2.5, Table 5	Coverage sheet of the matrix
Maintain a BibTeX bibliography	References	<code>literature-review/references.bib</code>
Download and version raw dataset	§3.2, Figure 1	<code>config.yaml</code> , <code>dvc.yaml</code> , <code>dvc.lock</code>
Check licensing and IRB	§3.5	<code>DATA_CARD.md</code> (Distribution, Collection)
Profile: shape, nulls, class balance	§3.4	<code>data/processed/profile.md</code> , <code>profile.json</code>
Write a data card	§3.6, Appendix A	<code>DATA_CARD.md</code>
Set up data versioning	§3.2	DVC remote on Cloudflare R2

2 Literature Review

2.1 Corpus and coding protocol

Search. We searched Google Scholar and arXiv and browsed the proceedings of security, machine-learning and NLP venues. Queries covered four themes:

- **Prompt injection:** “indirect prompt injection”, “prompt injection defense”, “prompt injection detection”;
- **RAG poisoning:** “RAG poisoning”, “knowledge corruption”, “retrieval corpus poisoning”;

- **Agents and MCP:** “MCP security”, “tool poisoning”, “LLM agent tool injection”;
- **Benchmarks:** “prompt injection benchmark”, “agent security benchmark”.

Selection. Results were screened on title and abstract, and the full text of each candidate was read before inclusion. A paper was included if it (i) studies indirect prompt injection or retrieval poisoning against an LLM application that uses retrieval, tools or MCP, as an attack, a defense, a benchmark or an evaluation method; (ii) was published or posted between 2023 and 2026; and (iii) is written in English with the full text available. arXiv preprints were eligible; their venue and year were verified before citing. Papers that did not meet all three criteria were excluded.

Corpus. The review set holds 20 papers. By primary contribution, 8 are defenses, 4 are benchmarks or evaluation environments, 3 are attacks, 1 is an evaluation-methodology paper, and 4 combine contributions (for example a benchmark and a defense). Twelve are rated highly relevant to our research question.

Each paper is coded as one row of a shared coding matrix. The frame records, among other dimensions:

- **Attack surface:** where adversarial content enters the system (RAG corpus, tool output at runtime, tool description or metadata, direct user input).
- **Payload:** how the content works on the model. *Instruction injection* issues commands; *content poisoning* is plausible false content that misleads by being believed, with no commands at all.
- **Attack objective:** what the attacker wants (task hijack, data exfiltration, misinformation, denial of service, tool-selection manipulation).
- **Defense class and enforcement point:** the defense mechanism and where in the stack it sits.
- **Primary metric, headline result, benchmarks used,** and whether the paper **addresses MCP explicitly**.

The payload dimension was added during coding: attack objective alone could not separate PoisonedRAG-style false passages from instructions that pursue the same misinformation goal. It corresponds one to one to the `attack_kind` column of our dataset (§3.3), so our results can be compared with prior work split the same way.

Verification. Coding was first drafted with an AI assistant from each paper’s abstract, introduction and threat-model section, then re-read and confirmed row by row by a team member. Venues and years were checked against publisher records (via DOI), DBLP and Semantic Scholar. This check found published versions of ten papers whose downloaded copies were arXiv preprints or carried no venue. The bibliography cites the published version, with the arXiv identifier kept as an `eprint` field.

2.2 Coverage of the reviewed set

The papers are spread unevenly across surfaces. Nine study several surfaces at once, five the RAG corpus, four tool outputs, and only two MCP tool descriptions [2], [4]. None studies direct user input, which is outside our scope. Thirteen papers concern instruction injection, four content

poisoning (all on RAG), and three both. Attack success rate (ASR) is the headline metric of 12 papers. Only three report a detection metric: AUROC [5] or true and false positive rates [6], [7].

2.3 Taxonomy

Table 2 organizes the papers by the threat they study or defend against; Table 3 organizes the defenses by mechanism.

Table 2: Threat taxonomy: attack surface by payload. Defense papers are placed by the threat they defend against.

Attack surface	Instruction injection	Content poisoning	Both
RAG corpus / retriever	—	RevPRAG [7], RAG traceback [8], adversarial passages [9], PoisonedRAG [10]	RobustRAG [11]
Tool output (runtime)	AgentDojo [12], CaMeL [13], Spotlighting [14], InjecAgent [15]	—	—
Tool description (MCP)	MCPTox [2]	—	ToolHijacker [4]
Several surfaces	SecAlign [16], StruQ [17], When Benchmarks Lie [5], Greshake et al. [1], PromptShield [6], Liu et al. [18], Jatmo [19], BIPIA [20]	—	MSB [3]

RAG corpus. Attacks inject a handful of passages into a knowledge base: PoisonedRAG reaches 90% ASR with five malicious texts per target question in a database of millions [10], and adversarial passages optimized against a dense retriever transfer to unseen corpora [9]. The RAG surface has the most varied defenses: an activation probe [7], a certified isolate-then-aggregate scheme [11] and post-hoc forensic traceback [8]. Every RAG paper in the set is about content poisoning, not injected instructions.

Tool output. Benchmarks place instructions in emails, web pages and documents that tools return: AgentDojo [12] and InjecAgent [15]. Defenses mark untrusted input in the prompt [14] or enforce control- and data-flow policies outside the model [13].

Tool description (MCP). MCPTox shows that tool-poisoning instructions placed in tool meta-data reach 72.8% ASR on o1-mini, with refusal rates under 3% across 20 agents [2]. ToolHijacker crafts tool documents that win tool selection without any access to the target model [4]. MSB benchmarks attacks at every stage of the MCP pipeline: tool signatures, parameters, responses and retrieval [3].

Several surfaces. Model-level defenses treat any data channel as untrusted: preference optimization [16], structured instruction tuning [17], and task-specific fine-tuning that removes instruction

following altogether [19]. Liu et al. formalize the attack space and show that none of ten defenses is sufficient [18].

Table 3: Defense taxonomy by mechanism.

Defense class	Papers	Enforcement point	Surfaces defended
Detection (classifier or probe)	When Benchmarks Lie [5], PromptShield [6], RevPRAG [7]	Prompt layer; runtime	Several; RAG
Input transformation / prompting	Spotlighting [14], BIPIA [20]	Prompt layer	Tool output; several
Training-time (fine-tuning, alignment)	SecAlign [16], StruQ [17], Jatmo [19]	Model weights	Several
Runtime policy / capabilities	CaMeL [13]	Orchestrator	Tool output
Certified / provable	RobustRAG [11]	Orchestrator	RAG
Forensics / traceback	RAG traceback [8]	Retriever / index	RAG
Architectural isolation	—	—	—

2.4 State-of-the-art benchmarks

Table 4 lists the benchmarks that the reviewed work evaluates on. PoisonedRAG is an attack paper, but its targeted questions serve as the de facto benchmark for RAG poisoning. Four of the eight benchmarks feed our dataset, and a fifth is our end-to-end harness.

Table 4: Benchmarks used by the reviewed work and their role in this project.

Benchmark	Surface	Payload	Scale	Metric	In our project
AgentDojo [12]	Tool output	Instruction	97 user tasks, 629 security test cases	ASR, utility	End-to-end harness (v0.1.35)
InjecAgent [15]	Tool output	Instruction	1,054 test cases, 17 user tools, 62 attacker tools	ASR	—
BIPIA [20]	Tool output (email, web, table, code, summaries)	Instruction	Five task types with text and code attacks	ASR	Training pool
Open-Prompt-Injection [18]	Several	Instruction	5 attacks, 10 defenses, 10 LLMs, 7 tasks	Attack success score	—
PromptShield [6]	Several	Instruction	Conversational and application data	TPR at low FPR	—

Benchmark	Surface	Payload	Scale	Metric	In our project
MCPTox [2]	MCP tool description	Instruction	1,312 cases, 45 servers, 353 tools	ASR	LOSO holdout
MSB [3]	MCP pipeline	Both	2,000 instances, 405 tools, 12 attack types	ASR, NRP	Transfer holdout
PoisonedRAG [10]	RAG corpus	Poisoning	NQ, HotpotQA, MS MARCO targets	ASR	Training pool

Three observations shape our evaluation design. First, nearly every benchmark measures *attack success against an agent*, not *detection of the injected text*, so detector results are rarely comparable across papers. Second, the detection papers that do exist warn that standard evaluation overstates performance. PromptShield argues that a deployed detector must operate at very low false positive rates, because benign traffic dominates; its detector reaches 65.3% TPR at 0.1% FPR against 9.4% for the best prior scheme [6]. Fomin finds that standard cross-validation reports pooled AUC 8.0–16.5 points higher than leave-one-dataset-out evaluation, and that 28–44% of a classifier’s top features are dataset shortcuts [5]. Third, the MCP benchmarks report that more capable models are often *more* susceptible [2], [3], so model scale is not a defense.

2.5 Gaps this project addresses

Table 5 crosses attack surface with defense class. An empty cell is a candidate gap in the reviewed set.

Table 5: Papers per attack surface and defense class. Architectural isolation (no papers) and direct user input (no papers) are omitted.

Surface	Detection	Prompting	Training	Runtime policy	Certified	Forensics	Attack or benchmark only
RAG corpus	1	0	0	0	1	1	2
Tool output	0	1	0	1	0	0	2
Tool description (MCP)	0	0	0	0	0	0	2
Several surfaces	2	1	3	0	0	0	3

We draw four gaps from the review. Each is stated relative to the reviewed set.

1. **No defense for MCP tool descriptions.** Two papers show that poisoned tool metadata hijacks agents [2], [4], and none defends against it. ToolHijacker reports that existing prevention and detection defenses fail on tool documents [4]. Neither of the two detectors coded as covering several surfaces addresses MCP [5], [6].

2. **No detector for tool outputs.** Tool-output defenses rewrite prompts [14] or enforce policy [13]; none classifies the returned text.
3. **No evaluation on a held-out surface.** The only held-out evaluation in the set leaves out whole *datasets* [5]. No paper trains a detector on some attack surfaces and tests it on another.
4. **Shortcut-blind detector evaluation.** Fomin shows that detectors learn dataset identity [5]; no other paper in the set reports a baseline that would reveal such shortcuts.

Our project addresses these gaps directly. The detector is trained on RAG passages and tool outputs and evaluated three ways: in distribution, on MCP tool descriptions held out entirely (leave-one-surface-out, gaps 1 and 3), and on MSB, a benchmark held out entirely (transfer). Tool outputs are part of training and of the in-distribution test (gap 2). Every detector will be reported next to a length-only baseline and at low false positive rates (gap 4, §3.4).

3 Data Acquisition

3.1 Sources and licensing

The corpus unifies four public benchmarks into one table of *untrusted text segments*. A segment is any text that reaches the model without the user writing it.

Table 6: Sources, roles and licenses. MSB contributes 55 adversarial and 21 benign tool descriptions and 15 adversarial tool outputs.

Source	Surface	Role	Adversarial	Benign	Groups	License
PoisonedRAG [10] + BEIR [21]	RAG corpus	Training pool	1,500	2,955	300	MIT; BEIR corpora per dataset
BIPIA [20]	Tool output	Training pool	2,806	2,806	2,806	MIT; TableQA CC BY-SA 4.0; NewsQA and XSum terms
MCPTox [2]	Tool description	LOSO holdout	485	362	45	None stated (team-internal copy)
MSB [3]	Tool description, tool output	Transfer holdout	70	21	11	MIT
Total			4,861	6,144		

AgentDojo 0.1.35 (MIT) is pinned as the end-to-end attack-success harness for Phase 3; it is not part of the table.

3.2 Download and versioning

Every upstream repository is pinned to an exact commit in `config.yaml`, so a rebuild fetches byte-identical inputs. The pipeline is a five-stage DVC graph (Figure 1). Two stages rebuild BIPIA contexts that BIPIA does not redistribute: summaries from XSum [22] and web QA from NewsQA

[23]. Each output is checked against pinned MD5 hashes. NewsQA sits behind terms of use, so it was built once with the official toolchain and is stored only in the team’s private remote.

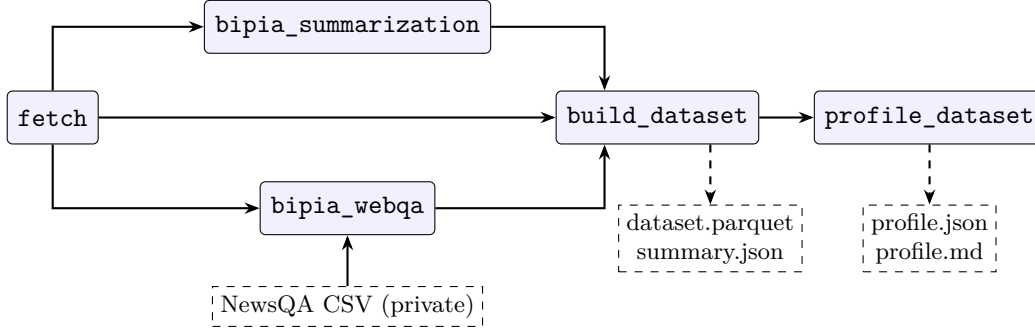


Figure 1: The DVC pipeline. `fetch` downloads the upstream repositories at the commits pinned in `config.yaml`. Solid arrows are stage dependencies; dashed arrows point to tracked outputs. All cached data is stored in a private Cloudflare R2 remote.

`dvc.lock` records the hash of every input and output. The current build (produced 2026-09-29, Git commit `af5ec37`) locks `dataset.parquet` at MD5 `a1c6426890678d576fb6231dc6050491`. Any team member can restore any committed version with `git checkout <commit>` followed by `make data`, and `dvc metrics diff` shows how row counts and the profile change in review.

3.3 Construction and cleaning

Schema and labels. Each row carries the text, a binary label (1 adversarial, 0 benign), the payload type (`attack_kind`: instruction or poisoning), the attack objective, the source’s own attack category verbatim, the source and subset, the attack surface, a group identifier, a split and a role. The objective is mapped from about 75 upstream categories in `config.yaml`, and changes to that mapping are reviewed like edits to the coding matrix.

Per-source construction. Adversarial RAG rows are PoisonedRAG’s five malicious texts for each of 100 target questions in NQ, MS MARCO and HotpotQA; benign rows are the top-10 real passages a Contriever retriever returns for the same question, which makes them hard negatives. For BIPIA, every clean context is a benign row, and one sampled attack per context, inserted at a sampled position (start, middle or end), is the adversarial row. MCPTox contributes its 485 poisoned tool descriptions and the clean descriptions of the same servers. MSB attack texts are assembled at run time upstream, so we rebuild the static text the model would see from MSB’s tool files and templates.

Splits and holdouts. Rows are split 70/15/15 into train, validation and test *by group* (a question, a context, an MCP server or an attack task), so a group never spans two splits. BIPIA’s own train/test boundary is kept, so its test split mostly holds attack families never seen in training. A `role` column assigns each source to the training pool or to a holdout. The data loader raises an error if fitting data would include a held-out source or the test split.

Cleaning. We removed formatting cues that would reveal the label: surrounding whitespace (an artifact of BIPIA’s insertion), passage titles that only benign PoisonedRAG rows would have had, and source-specific tool-description layouts. All tool descriptions now share one format. 47 exact

duplicate texts with the same label were dropped. The build stops, writing nothing, if a text appears with both labels, if a group spans splits, if a benign row carries attack fields, or if a category lacks an objective mapping.

3.4 Profile

The `profile_dataset` stage recomputes the profile whenever the dataset changes.

Shape and nulls. The table has 11,005 rows and 12 columns. The only nulls are the three attack fields on the 6,144 benign rows, which the schema requires. No other column contains a null or a blank text, and there are no duplicate IDs or texts.

Class balance. The corpus is 44.2% adversarial, and the share is stable across splits. It varies strongly by source and role, however (Table 7).

Table 7: Class balance by split, role and source.

Group	Adversarial	Benign	Adversarial share
All rows	4,861	6,144	44.2%
Train split	3,469	4,393	44.1%
Validation split	713	875	44.9%
Test split	679	876	43.7%
Training pool	4,306	5,761	42.8%
PoisonedRAG (RAG corpus)	1,500	2,955	33.7%
BIPIA (tool output)	2,806	2,806	50.0%
LOSO holdout: MCPTox	485	362	57.3%
Transfer holdout: MSB	70	21	76.9%

Because MSB is 76.9% adversarial, a detector that flags everything scores 77% accuracy on it. We therefore report AUROC and the true positive rate at a fixed low false positive rate, not accuracy.

A length shortcut. Text length differs sharply between classes, in opposite directions on different sources (Table 8). Poisoned passages are shorter than retrieved ones; injected tool descriptions are longer than clean ones.

Table 8: Median text length in characters, by source and label.

Source	Benign	Adversarial
PoisonedRAG	389	183
BIPIA	1,752	1,816
MCPTox	88	292
MSB	156	314

We measure how much length alone explains with two numbers (Table 9): the AUROC of a logistic regression on log length, fit on the training pool, and length separability, $\max(\text{AUROC}, 1 - \text{AUROC})$ of raw length, which needs no training and counts a cue in either direction.

Table 9: What text length alone achieves on each evaluation set. The fitted model learns the wrong direction for PoisonedRAG, which separability exposes.

Evaluation set	Fitted AUROC	Separability
In-distribution test	0.491	0.509
BIPIA only	0.532	0.532
PoisonedRAG only	0.138	0.862
LOSO holdout (MCPTox)	0.774	0.774
Transfer holdout (MSB)	0.866	0.866

Length alone reaches 0.77 AUROC on the LOSO holdout and 0.87 on the transfer holdout. A detector scoring near those values may be classifying length, not injection. This is the kind of shortcut Fomin warns about [5], and it is why every detector in Phase 3 will be reported next to this baseline.

3.5 Licensing, privacy and IRB

Licensing. PoisonedRAG, BIPIA and MSB are MIT-licensed; BIPIA’s TableQA contexts are CC BY-SA 4.0. NewsQA and the CNN stories it draws on are available under terms of use that forbid redistribution, and XSum’s BBC articles are licensed for research use. MCPTox states no license, so its use is team-internal. The data therefore stays in a private remote; if it is ever shared, we will share the build pipeline, not the rebuilt rows.

Privacy. A spot-check of every benign BIPIA email and NewsQA context (samples plus a pattern scan for email addresses, phone numbers, social security and card numbers) found no full account or card numbers, phone numbers or SSNs. It did find that BIPIA’s emails, which come from OpenAI Evals, are not synthetic: they read as one company’s real inbox, with names, vendors, amounts and masked account numbers from 2022. The news contexts name public figures and, in crime and accident reports, private individuals. This content is already public through the upstream benchmarks. We keep it unchanged, and we will not quote email rows in our write-ups.

IRB. The project does not interact with people or collect data from them; every row comes from publicly released benchmarks. Under the U.S. Common Rule (45 CFR 46.102) this is generally not human subjects research, so no IRB review was sought. If the work is published beyond the course, we will request a formal “not human subjects research” determination from the institution’s IRB.

3.6 Data card

The corpus is documented in `DATA_CARD.md` at the repository root. It opens with a summary and per-source table in the style of Data Cards [24], then answers the questions of Datasheets for

Datasets [25] on motivation, composition, collection, preprocessing, uses, distribution and maintenance. Appendix A reproduces it in full. Besides the facts in this section, the card records the intended uses and the uses to avoid: training on held-out sources, measuring direct-injection robustness, estimating false positive rates on MCP tool responses (no benign ones exist), comparing payload types without stratifying, and building attack generators.

4 Summary and Next Steps

The review maps 20 papers onto a taxonomy of attack surface, payload and defense class. Within that set, MCP tool descriptions have no defense, tool outputs have no detector, and no detector has been evaluated on an unseen attack surface. The dataset is built to measure exactly those cases. It is versioned end to end, split by group with two sources fully held out, profiled, and documented.

Phase 3 will train detector baselines on the training pool and report, for each:

- AUROC and TPR at a low fixed FPR on the in-distribution test set, the LOSO holdout (MCPTox) and the transfer holdout (MSB);
- results stratified by `attack_kind`, because the training pool mixes poisoning and instruction injection while both holdouts are instruction-only;
- the length-only baseline alongside, as the score to beat.

Experiments are tracked in the team’s shared MLflow server. AgentDojo will measure whether a detector reduces end-to-end attack success in a defended agent.

Use of AI assistance. The coding of all 20 papers was first drafted with an AI assistant (Claude) and then confirmed by a team member, and venues were checked against publisher and index records. Parts of the data tooling were written with AI assistance and merged through reviewed pull requests. All analysis choices and the text of this report were reviewed by the team.

A Data card

This data card describes the corpus built for the project. It opens with a summary and a per-source table in the style of Pushkarna et al., *Data Cards* (FAccT 2022), then answers the questions of Gebru et al., *Datasheets for Datasets* (CACM 2021), section by section.

Numbers describe the build recorded in `data/processed/summary.json`, `data/processed/profile.md` and `dvc.lock`.

A.1 At a glance

Task	Binary classification of untrusted text segments: 1 adversarial (prompt injection or poisoning), 0 benign
Unit	One text segment that reaches an LLM without the user writing it: a retrieved passage, a fetched document, an MCP tool description or a tool response
Size	11,005 rows: 4,861 adversarial, 6,144 benign (47 exact duplicates dropped)
Surfaces	<code>rag_corpus</code> , <code>tool_output</code> , <code>tool_description</code>

Sources	PoisonedRAG (+ BEIR), BIPIA, MCPTox, MSB
Held out	MCPTox (<code>loso_holdout</code> , an unseen surface) and MSB (<code>transfer_holdout</code> , an unseen benchmark) are never trained on
File	<code>data/processed/dataset.parquet</code> , built by the <code>build_dataset</code> DVC stage
Load with	<code>src/data/dataset.py</code> , which raises <code>HoldoutError</code> when a fit would include held-out data
Profile	<code>data/processed/profile.md</code> : $11,005 \times 12$, no unexpected nulls, no duplicate ids or texts, 44.2% adversarial
Design	docs/superpowers/specs/2026-09-29-build-dataset-design.md

A.1.1 Sources

Source	Surface	Role	Adversarial	Benign	Groups	Upstream license
PoisonedRAG + BEIR	<code>rag_corpus</code>	<code>train_pool</code>	1,500	2,955	300	MIT (BEIR corpora: per-dataset terms)
BIPIA	<code>tool_output</code>	<code>train_pool</code>	2,806	2,806	2,806	MIT; TableQA CC BY-SA 4.0; NewsQA and XSum terms
MCPTox	<code>tool_description</code>	<code>loso_holdout</code>	485	362	45	None stated (team-internal copy)
MSB	<code>tool_description</code>	<code>transfer_holdout</code>	55	21	11 (with tool output)	MIT
MSB	<code>tool_output</code>	<code>transfer_holdout</code>	15	0		MIT

Each upstream repository is pinned to a commit in `config.yaml`.

A.2 Motivation

For what purpose was the dataset created? To train and evaluate detectors for *indirect* prompt injection: classifying text that an LLM application reads from RAG retrieval or MCP tools as adversarial or benign. Detection had to be measurable in three settings: within a surface, on an unseen surface (leave-one-surface-out, MCP tool descriptions), and on an unseen benchmark (MSB). No single public source covers all three, so the sources are unified into one schema with fixed holdouts.

Who created the dataset, and on behalf of which entity? The CS 7267 (Machine Learning) project team.

- Carter Corbin, Max Edmonds, Mohammed Adil Ahmed, Oluwamayowa Adewummi, Adedapo Odeyemi

Who funded the creation of the dataset?

- None

A.3 Composition

What do the instances represent? Each row is one untrusted text segment with its label and provenance. Columns:

Column	Meaning
id	Unique; traceable to the source record (for example <code>mcptox:FileSystem_1</code>)
text	Detector input, stripped of surrounding whitespace
label	1 adversarial, 0 benign
attack_kind	<code>instruction</code> (injected commands) or <code>poisoning</code> (false content); null if benign
attack_objective	<code>task_hijack</code> , <code>data_exfiltration</code> , <code>misinformation</code> , <code>denial_of_service</code> , <code>tool_selection_manipulation</code> ; null if benign
attack_family	The source's own attack category, verbatim; null if benign
source, subset	Origin dataset and its sub-collection (for example <code>bipia</code> / <code>email</code>)
attack_surface	<code>rag_corpus</code> , <code>tool_output</code> or <code>tool_description</code>
group	Unit that decides the split: question, context, MCP server or attack task
split	<code>train</code> , <code>val</code> or <code>test</code> , assigned per group
role	<code>train_pool</code> , <code>loso_holdout</code> or <code>transfer_holdout</code>

How many instances are there? 11,005 in total. Adversarial rows by payload and objective:

attack_kind	attack_objective	Rows
instruction	task_hijack	2,847
instruction	data_exfiltration	214
instruction	denial_of_service	149
instruction	misinformation	129
instruction	tool_selection_manipulation	22
poisoning	misinformation	1,500

Is it a sample from a larger set? Yes, in three places:

- **PoisonedRAG:** 100 target questions each from NQ, MS MARCO and HotpotQA. Benign rows are each question's top-10 Contriever-retrieved real passages (hard negatives), not a random corpus sample.

- **BIPIA:** one attack per context, sampled with seed 7267 and inserted at a sampled position (start, middle or end), rather than BIPIA’s full ~600k context $\times$ attack cross product.
- **MSB:** only the static text an attack produces. Variants whose attack exists only at run time (most `name_overlap` and `tool_transfer` files reuse a clean docstring) are left out.

What data does each instance consist of? Raw text only: no features, embeddings or tokenization.

Is there a label or target? `label`, `attack_kind`, `attack_objective` and `attack_family` support per-category analysis. `attack_objective` is the team’s coding judgment, mapped from each source’s own categories in `config.yaml`. It is not an upstream label.

Is any information missing from individual instances? Benign rows have null attack fields by design: 6,144 nulls in each of `attack_kind`, `attack_objective` and `attack_family`, all on benign rows. No other column has a null or a blank text (`data/processed/profile.md`, regenerated by the `profile_dataset` stage). MSB contributes no benign `tool_output` rows because upstream has none.

Are relationships between instances explicit? Yes, through `group`. An adversarial row and the benign context it was built from share a group (BIPIA, MSB descriptions), as do a question’s poisoned and retrieved passages (PoisonedRAG).

Are there recommended splits? Yes. Splits are 70/15/15 by group, so a group never spans two splits. BIPIA keeps its own train/test boundary (our `val` is 15% of BIPIA train groups), so BIPIA `test` mostly holds attack families never seen in training. `role` sits on top of `split`:

Role	Train	Val	Test	Use
<code>train_pool</code>	7,217	1,416	1,434	Fit on train, tune on val, report in-distribution on test
<code>loso_holdout</code>	572	166	109	Evaluation only; score every row
<code>transfer_holdout</code>	73	6	12	Evaluation only; score every row

Splits inside the holdouts exist only for consistency. Evaluation should use the whole source.

Are there errors, noise or redundancies?

- 47 exact duplicate texts with the same label were dropped (first kept). The build fails if a text appears with both labels.
- Near-duplicates are not merged, for example the same BIPIA email with an extra header, or MCPTox attacks built from shared templates. Grouping keeps most of them inside one split.
- BIPIA WebQA contexts were rebuilt from NewsQA with the official toolchain. They match our pinned md5s but not BIPIA’s own md5s, so story text can differ slightly from upstream’s build.

Known shortcuts (spurious cues a detector could learn). Already removed: surrounding whitespace (BIPIA insertion artifacts), passage titles that only benign PoisonedRAG

rows would have had, and source-specific tool-description layouts (all rendered as `Tool: {name}\nDescription: {description}`).

Still present, so check before trusting a score. Median character lengths:

Source	Benign	Adversarial
PoisonedRAG	389	183
BIPIA	1,752	1,816
MCPTox	88	292
MSB	156	314

Length alone separates the classes well, and in opposite directions: poisoned passages are shorter than retrieved ones, while injected tool descriptions are longer than clean ones. Length separability, $\max(\text{AUROC}, 1 - \text{AUROC})$ of raw length, and the AUROC of `LengthBaseline` (logistic regression on log length, fit on `train_pool` train):

Eval set	Fitted AUROC	Separability
In-distribution test	0.491	0.509
BIPIA	0.532	0.532
PoisonedRAG	0.138	0.862
LOSO (MCPTox)	0.774	0.774
Transfer (MSB)	0.866	0.866

Report both next to every detector, so length is the score to beat. To regenerate, run `PYTHONPATH=src/data:src/eval uv run python src/models/baselines.py`.

Is the dataset self-contained? No. `dataset.parquet` is derived from upstream repositories pinned in `config.yaml`, BEIR corpora, XSum (Hugging Face) and NewsQA. NewsQA and the CNN stories sit behind terms of use, so that build lives only in the team’s private DVC remote.

Does the dataset contain confidential data? Nothing secret, but the BIPIA emails come from a real inbox. All text comes from public benchmarks. A spot-check on 2026-10-01 (samples plus a regex scan of every benign BIPIA email and NewsQA row for email addresses, phone numbers, SSNs and card numbers) found:

- **BIPIA emails (78 benign contexts) do not look synthetic.** BIPIA takes them from [OpenAI Evals](#) and does not say whether they are real. They read as one company’s actual inbox: 64 of the 78 are addressed to “David” at Moonchaser (`david@moonchaser.io`). Most are Mercury bank notifications, plus receipts and invoices from Deel, PayPal, Upwork, Webflow, Eurostar and Air Canada, with real dates in 2022, dollar amounts and order numbers. Account numbers are masked to their last four digits (three accounts appear). No full account or card numbers, phone numbers or SSNs were found.

- **NewsQA contexts are published CNN articles.** The only contact details are CNN’s own (a fact-check address and a public phone line).

Does it contain offensive, insulting or threatening content? Yes, by construction. Adversarial rows include working prompt-injection payloads, malware and ransomware instructions (BIPIA code attacks), scam, phishing and propaganda text, and data-exfiltration instructions aimed at agents.

Does it identify individuals or contain sensitive personal data? Yes, in two ways:

- **News-derived contexts** (NewsQA, XSum, BEIR passages) name people as reported in published articles. Most are public figures, but crime and accident reports also name private individuals.
- **BIPIA emails** name private individuals in a business setting: the recipient, their company, and payees or senders such as contractors. They also reveal that company’s vendors, payment amounts and transaction dates. This is financial information about identifiable people, already public through OpenAI Evals and BIPIA.

Nothing was collected from people by this project, and no rows were removed. Do not quote BIPIA email rows in the paper or slides; use a BIPIA table or code example to illustrate `tool_output`.

A.4 Collection process

How was the data acquired? Entirely from existing datasets; nothing was newly written by annotators.

1. `fetch` downloads pinned files from each upstream repository, plus the BEIR passages PoisonedRAG targets.
2. `bipia_summarization` and `bipia_webqa` rebuild BIPIA contexts that upstream does not redistribute (XSum, NewsQA).
3. `build_dataset` converts every source to the schema, assigns splits, validates invariants and deduplicates.

The pipeline is defined in `dvc.yaml`. `dvc.lock` records the hash of every input and output.

What sampling strategy was used? See *Is it a sample* above. All randomness uses seed 7267, and the same seed produces an identical file (tested).

Who was involved, and how were they compensated? The project team, as coursework. Upstream authors created all underlying text.

Over what timeframe? The build was designed and produced on 2026-09-29. The current `dvc.lock` was committed then in `af5ec37` (“Fix label shortcuts and split leaks found in review”) and locks `dataset.parquet` at md5 `a1c6426890678d576fb6231dc6050491`. Upstream commits are pinned in `config.yaml`. The text itself is older: the BIPIA emails date from 2022, and the news articles are older still.

Were ethical review processes conducted? No IRB review was sought. The project does not interact with people or collect data from them: every row comes from existing, publicly released

benchmarks. Under the U.S. Common Rule (45 CFR 46.102), that is generally not human subjects research, because no data is obtained through interaction with people and publicly available information is not “private information.”

The closest case is the BIPIA emails, which identify private individuals (see *Composition*). They were already published through OpenAI Evals and BIPIA, and this project uses them unchanged, only as model inputs, without studying the people in them.

- The project team will engage with the University’s IRB for a “not human subjects research” determination if this work is published beyond the course.

Consent and notification of individuals: no data was collected from people, so this project sought no consent and sent no notifications. The people in BIPIA’s emails and in the news articles were not asked by this project; whether they consented upstream is not documented.

A.5 Preprocessing, cleaning and labeling

What preprocessing was done?

- Whitespace stripped from every `text`.
- BIPIA attacks inserted into contexts following `bipia/data/utils.py` (`\n` joins at start or end, a Punkt sentence boundary for middle). Code contexts joined with `\n`.
- MSB docstrings extracted with `ast` without importing upstream code. Prompt-injection descriptions and tool responses rebuilt from MSB’s templates, with a fixed `{pid}`.
- Exact duplicates dropped. Rows that fail any invariant stop the build.

How were labels assigned? `label` comes from each source’s construction: an attack text is adversarial, and a clean context or tool description is benign. `attack_objective` is a manual mapping of about 75 upstream categories, reviewed like an edit to the team’s literature-review coding matrix.

Is the raw data saved? Yes. Fetched and rebuilt inputs are tracked by DVC under `data/raw` and `data/generated`.

Is the preprocessing software available? Yes: `src/data/`, tested in `tests/test_build_dataset.py` and `tests/test_generate_bipia.py`.

A.6 Uses

Intended uses

- Training and evaluating text-level detectors for indirect prompt injection on RAG and MCP surfaces.
- Measuring generalization to an unseen surface (MCPTox) and an unseen benchmark (MSB).
- Per-category error analysis by `attack_kind`, `attack_objective` and `attack_family`.

Uses to avoid

- **Training on held-out sources.** Fitting on MCPTox or MSB invalidates the LOSO and transfer results. Always load through `dataset.load("fit", ...)`.

- **Measuring direct-injection or jailbreak robustness.** No row comes from the user’s own message.
- **Estimating false-positive rates on MCP tool responses.** There are no benign MCP tool outputs.
- **Reporting accuracy on the holdouts.** Class balance varies by source: MSB is 76.9% adversarial and MCPTox 57.3%, so labeling everything adversarial already scores 77% on MSB. Report AUROC and TPR at a fixed low FPR instead.
- **Comparing results across payload types without stratifying.** Training mixes poisoning and instruction injection, but both holdouts are instruction-only. Report scores per `attack_kind`.
- **Building or tuning attack generators.** The adversarial rows are working payloads.

Could the composition affect future uses? Benign examples come from a narrow set of domains: news, Wikipedia-style QA passages, synthetic emails, code snippets, and the tool descriptions of a few dozen MCP servers. A detector may flag ordinary text from other domains, or text that merely *talks about* instructions (documentation, tutorials). False-positive rates outside these domains are unknown.

A.7 Distribution

Will the dataset be distributed outside the team?

- No. The data lives in a private Cloudflare R2 DVC remote, and teammates get access keys from the project owner.

Constraints if that changes:

- NewsQA and the CNN stories may not be redistributed. Share the build pipeline, not the rebuilt contexts.
- MCPTox states no license upstream, so its use here is team-internal only.
- TableQA content is CC BY-SA 4.0, so derivatives must keep that license.
- Export controls or other regulatory restrictions: None known.

A.8 Maintenance

Who maintains the dataset, and how can they be contacted?

- Adedapo Odeyemi ([@blacng](#)), for the CS7267 project team. Open an issue on this repository for questions or problems with the data.

Will the dataset be updated? Through pull requests. Changes to pinned source commits, sampling parameters or the objective mapping go in their own PR, with `dvc.lock` and `summary.json` committed and the rebuilt data pushed. `uv run dvc metrics diff` shows how row counts changed.

Are older versions kept? Yes. Every committed `dvc.lock` points to its outputs in the DVC remote, so `git checkout <commit> && make data` restores that version.

Can others contribute? Team members, through the pull-request workflow in [README.md](#).

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-Integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, in CCS ’23. ACM, Nov. 2023, pp. 79–90. doi: [10.1145/3605764.3623985](#).
- [2] Z. Wang *et al.*, “MCPTox: A benchmark for tool poisoning on real-world MCP servers,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 42, pp. 35811–35819, Mar. 2026, doi: [10.1609/aaai.v40i42.40895](#).
- [3] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu, “MCP security bench (MSB): Benchmarking attacks against model context protocol in LLM agents,” in *The fourteenth international conference on learning representations (ICLR)*, 2026. Available: <https://arxiv.org/abs/2510.15994>
- [4] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Gong, and L. Sun, “Prompt injection attack to tool selection in LLM agents,” in *Proceedings 2026 network and distributed system security symposium*, in NDSS 2026. Internet Society, 2026. doi: [10.14722/ndss.2026.230675](#).
- [5] M. Fomin, “When benchmarks lie: Evaluating malicious prompt classifiers under true distribution shift.” AIWILD Workshop at ICLR 2026, 2026. Available: <https://arxiv.org/abs/2602.14161>
- [6] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, “PromptShield: Deployable detection for prompt injection attacks,” in *Proceedings of the fifteenth ACM conference on data and application security and privacy*, in CODASPY ’25. ACM, Jun. 2025, pp. 341–352. doi: [10.1145/3714393.3726501](#).
- [7] X. Tan, H. Luan, M. Luo, X. Sun, P. Chen, and J. Dai, “RevPRAG: Revealing poisoning attacks in retrieval-augmented generation through LLM activation analysis,” in *Findings of the association for computational linguistics: EMNLP 2025*, Association for Computational Linguistics, 2025, pp. 12999–13011. doi: [10.18653/v1/2025.findings-emnlp.698](#).
- [8] B. Zhang *et al.*, “Traceback of poisoning attacks to retrieval-augmented generation,” in *Proceedings of the ACM on web conference 2025*, in WWW ’25. ACM, Apr. 2025, pp. 2085–2097. doi: [10.1145/3696410.3714756](#).
- [9] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, “Poisoning retrieval corpora by injecting adversarial passages,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, Association for Computational Linguistics, 2023, pp. 13764–13775. doi: [10.18653/v1/2023.emnlp-main.849](#).
- [10] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *34th USENIX security symposium (USENIX security 25)*, USENIX Association, 2025, pp. 3827–3844. Available: <https://arxiv.org/abs/2402.07867>
- [11] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, “Certifiably robust RAG against retrieval corruption,” in *2026 IEEE conference on secure and trustworthy machine learning (SaTML)*, IEEE, Mar. 2026, pp. 392–410. doi: [10.1109/satml68715.2026.00029](#).

- [12] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in neural information processing systems 37*, in NeurIPS 2024. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024, pp. 82895–82920. doi: [10.52202/079017-2636](https://doi.org/10.52202/079017-2636).
- [13] E. Debenedetti *et al.*, “Defeating prompt injections by design,” in *2026 IEEE conference on secure and trustworthy machine learning (SaTML)*, IEEE, Mar. 2026, pp. 587–618. doi: [10.1109/satml68715.2026.00040](https://doi.org/10.1109/satml68715.2026.00040).
- [14] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kıcıman, “Defending against indirect prompt injection attacks with spotlighting,” in *Proceedings of the conference on applied machine learning in information security (CAMLIS 2024)*, in CEUR workshop proceedings. 2024. Available: <https://arxiv.org/abs/2403.14720>
- [15] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the association for computational linguistics ACL 2024*, Association for Computational Linguistics, 2024, pp. 10471–10506. doi: [10.18653/v1/2024.findings-acl.624](https://doi.org/10.18653/v1/2024.findings-acl.624).
- [16] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, “SecAlign: Defending against prompt injection with preference optimization,” in *Proceedings of the 2025 ACM SIGSAC conference on computer and communications security*, in CCS ’25. ACM, Nov. 2025, pp. 2833–2847. doi: [10.1145/3719027.3744836](https://doi.org/10.1145/3719027.3744836).
- [17] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “StruQ: Defending against prompt injection with structured queries,” in *34th USENIX security symposium (USENIX security 25)*, USENIX Association, 2025, pp. 2383–2400. Available: <https://arxiv.org/abs/2402.06363>
- [18] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX security symposium (USENIX security 24)*, USENIX Association, 2024. Available: <https://arxiv.org/abs/2310.12815>
- [19] J. Piet *et al.*, “Jatmo: Prompt injection defense by task-specific finetuning,” in *Computer security – ESORICS 2024*, Springer Nature Switzerland, 2024, pp. 105–124. doi: [10.1007/978-3-031-70879-4_6](https://doi.org/10.1007/978-3-031-70879-4_6).
- [20] J. Yi *et al.*, “Benchmarking and defending against indirect prompt injection attacks on large language models,” in *Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining v.1*, in KDD ’25. ACM, Jul. 2025, pp. 1809–1820. doi: [10.1145/3690624.3709179](https://doi.org/10.1145/3690624.3709179).
- [21] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Proceedings of the neural information processing systems track on datasets and benchmarks*, 2021. Available: <https://arxiv.org/abs/2104.08663>
- [22] S. Narayan, S. B. Cohen, and M. Lapata, “Don’t give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization,” in *Proceedings of the 2018 conference on empirical methods in natural language processing*, Association for Computational Linguistics, 2018, pp. 1797–1807. doi: [10.18653/v1/D18-1206](https://doi.org/10.18653/v1/D18-1206).

- [23] A. Trischler *et al.*, “NewsQA: A machine comprehension dataset,” in *Proceedings of the 2nd workshop on representation learning for NLP*, Association for Computational Linguistics, 2017, pp. 191–200. doi: [10.18653/v1/W17-2623](https://doi.org/10.18653/v1/W17-2623).
- [24] M. Pushkarna, A. Zaldivar, and O. Kjartansson, “Data cards: Purposeful and transparent dataset documentation for responsible AI,” in *2022 ACM conference on fairness, accountability, and transparency (FAccT ’22)*, ACM, 2022, pp. 1776–1826. doi: [10.1145/3531146.3533231](https://doi.org/10.1145/3531146.3533231).
- [25] T. Gebru *et al.*, “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021, doi: [10.1145/3458723](https://doi.org/10.1145/3458723).